

Blockchain Experiment:11

Aim - To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory -

Hyperledger Fabric Network Setup

- Consortium & Membership Service Provider (MSP): Fabric is permissioned, meaning only authorized organizations can join. MSPs define identities and roles.
- Peers: Nodes that maintain the ledger and endorse transactions. Each organization typically runs peers.
- Orderer: The ordering service packages endorsed transactions into blocks and delivers them to peers.
- Ledger: Composed of a blockchain (immutable transaction log) and a state database (current values).
- Channels: Private sub-networks where a subset of organizations transact confidentially.

Chaincode (Smart Contracts)

- Chaincode defines the business logic for asset management.
- Written in Go, Java, or Node.js.
- Deployed to peers, then instantiated on a channel.
- Functions typically include createAsset, queryAsset, transferAsset, and deleteAsset.

Transaction Flow

1. Proposal Creation: A client application submits a transaction proposal (e.g., create asset).
2. Endorsement: Peers simulate the transaction using chaincode and return signed results.
3. Ordering: The orderer sequences endorsed transactions into blocks.
4. Validation & Commit: Peers validate endorsements, check for conflicts, and commit the block to the ledger.

Asset Operations

- Asset Creation:
 - Chaincode function createAsset stores asset attributes (ID, owner, value) in the ledger's state database.
- Asset Querying:
 - queryAsset retrieves asset details by key. Rich queries are possible if CouchDB is used as the state database.
- Asset Transfer:
 - transferAsset updates the owner field of an asset. The transaction is endorsed, ordered, and committed, ensuring immutability and consensus.

Key Principles

- Immutability: Once committed, transactions cannot be altered.

- Consensus: Endorsement policies ensure multiple organizations agree before a transaction is valid.
- Privacy: Channels and private data collections allow selective visibility.
- Trust: Identities are cryptographically verified via MSPs.

In essence, Hyperledger Fabric provides a modular, permissioned blockchain framework where chaincode governs asset lifecycles. The theory emphasizes identity-driven trust, modular architecture, and consensus-based validation — ensuring secure creation, querying, and transfer of assets across organizations.

Prerequisite: Check if WSL is Installed

Open **PowerShell (Run as Administrator)** and run:

```
wsl --list --verbose
```

If it shows **no distributions**, you must install Ubuntu.

Install Ubuntu for WSL

Run this command in **PowerShell (Admin)**:

```
wsl --install -d Ubuntu
```

This installs:

- Ubuntu
- WSL2 kernel

The installation may take a few minutes.

Restart Your Computer

After installation finishes, **restart your PC**.

Open Ubuntu

Open:

Start Menu → Ubuntu

It will ask you to create:

Username

Password

Now Ubuntu is ready.

User myuser

Pass : user123

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> wsl --install
>>
Ubuntu is already installed.
Launching Ubuntu...
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Fri Apr  3 12:42:09 UTC 2026

System load:  0.83               Processes:            71
Usage of /:   0.1% of 1006.85GB   Users logged in:     0
Memory usage: 5%                IPv4 address for eth0: 172.25.13.113
Swap usage:   0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge
```

```
Swap usage: 0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/root/.hushlogin file.
root@snehal:~# adduser myuser
info: Adding user `myuser' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `myuser' (1000) ...
info: Adding new user `myuser' (1000) with group `myuser (1000)' ...
info: Creating home directory `/home/myuser' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for myuser
Enter the new value, or press ENTER for the default
    Full Name []: snehal
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
info: Adding new user `myuser' to supplemental / extra groups `users' ...
info: Adding user `myuser' to group `users' ...
root@snehal:~#
```

```
Is the information correct? [Y/n] y
info: Adding new user `myuser' to supplemental / extra groups `users' ...
info: Adding user `myuser' to group `users' ...
root@snehal:~# usermod -aG sudo myuser
root@snehal:~# su - myuser
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Fri Apr  3 12:46:19 UTC 2026

System load:  0.02                Processes:            31
Usage of /:   0.1% of 1006.85GB   Users logged in:     1
Memory usage: 5%                 IPv4 address for eth0: 172.25.13.113
Swap usage:   0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/myuser/.hushlogin file.
myuser@snehal:~$ █
```

PHASE 1 — SYSTEM SETUP

Step 1: Update system

code:

```
sudo apt update
```

```
sudo apt install curl build-essential git -y
```

Step 2: Install Go 1.18.10

code:

```
wget https://go.dev/dl/go1.18.10.linux-amd64.tar.gz
```

```
sudo rm -rf /usr/local/go && sudo tar -C /usr/local -xzf go1.18.10.linux-amd64.tar.gz
```

```
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.code:rc
```

```
echo 'export GOPATH=$HOME/go' >> ~/.code:rc
```

```
echo 'export PATH=$PATH:$GOPATH/bin' >> ~/.code:rc
```

```
source ~/.code:rc
```

```
go version
```

```
[~u user] file ...
anushka@Anushka:~/private_ethereum_setup$ sudo apt
[sudo] password for anushka:
apt 2.8.3 (amd64)
Usage: apt [options] command

apt is a commandline package manager and provides commands for
searching and managing as well as querying information about packages.
It provides the same functionality as the specialized APT tools,
like apt-get and apt-cache, but enables options more suitable for
interactive use by default.

Most used commands:
  list - list packages based on package names
  search - search in package descriptions
  show - show package details
  install - install packages
  reinstall - reinstall packages
  remove - remove packages
  autoremove - automatically remove all unused packages
  update - update list of available packages
  upgrade - upgrade the system by installing/upgrading packages
  full-upgrade - upgrade the system by removing/installing/upgrading packages
  edit-sources - edit the source information file
  satisfy - satisfy dependency strings

See apt(8) for more information about the available commands.
Configuration options and syntax is detailed in apt.conf(5).
Information about how to configure sources can be found in sources.list(5).
Package and version choices can be expressed via apt_preferences(5).
Security details are available in apt-secure(8).
                                This APT has Super Cow Powers.
anushka@Anushka:~/private_ethereum_setup$
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libcurl3t64-gnutls libcurl4t64
The following packages will be upgraded:
  curl libcurl3t64-gnutls libcurl4t64
3 upgraded, 0 newly installed, 0 to remove and 220 not upgraded.
Need to get 902 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 curl amd64 8.5.0-2ubuntu10.8 [227 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libcurl4t64 amd64 8.5.0-2ubuntu10.8 [334 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libcurl3t64-gnutls amd64 8.5.0-2ubuntu10.8 [334 kB]
Fetched 902 kB in 2s (402 kB/s)
(Reading database ... 40788 files and directories currently installed.)
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bzip2 cpp cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg dpkg-dev fakeroot g++ g++-13
  g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu
  gcc-14-base gcc-x86-64-linux-gnu libalgorithm-diff-perl libalgorithm-diff-xs-perl
  libalgorithm-merge-perl libaom3 libasan8 libatomic1 libc-bin libc-dev-bin libc-devtools libc6
  libc6-dev libcc1-0 libcrypt-dev libde265-0 libdpkg-perl libfakeroot libfile-fcntllock-perl
  libgcc-13-dev libgcc-s1 libgd3 libgomp1 libheif-plugin-aomdec libheif-plugin-aomenc
  libheif-plugin-libde265 libheif1 libhwasan0 libisl23 libitm1 liblsan0 libmpc3 libquadmath0
  libstdc++-13-dev libstdc++6 libtsan2 libubsan1 libxpm4 linux-libc-dev locales lto-disabled-list make
  manpages-dev rpcsvc-proto
Suggested packages:
```

```
myuser@snehal:~$ curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
```

```
2026-04-03T13:00:22Z INFO Waiting for automatic snapd restart...
go (1.18/stable) 1.18.10 from Canonical✓ installed
```

Install weftility

code:

```
npm install -g @hyperledger/weftility
```

Verify:

code:

```
weft --version
```

Expected: a version number, no errors.

```
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Consider
  out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is
  h more comprehensive and powerful.
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized
  curity vulnerabilities, which have been fixed in the current version. Please update. Support for old
  sions may be purchased (at exorbitant rates) by contacting i@izs.me

added 158 packages in 34s

22 packages are looking for funding
  run `npm fund` for details
```

Step 3: Install Docker Compose v1.29.2

code:

```
sudo curl -L
```

```
"https://github.com/docker/compose/releases/download/1.29.2/docker-compose-$(uname
-s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

```
sudo chmod +x /usr/local/bin/docker-compose
```

```
docker-compose version
```

Expected: docker-compose version 1.29.2

Before continuing, open Docker Desktop on Windows → Settings → Resources → WSL Integration → Enable for your Ubuntu distro → Apply & Restart. Then verify:

code:

```
docker version
```

```
anushka@Anushka:/mnt/c/Users/anush$ sudo apt install docker.io -y
[sudo] password for anushka:
Sorry, try again.
[sudo] password for anushka:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libdrm-nouveau2 libdrm-radeon1 libgl1-amber-dri libglapi-mesa libllvm17t64 libwayland-server0 libxcb-dri2-0
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base iptables libip4tc2 libip6tc2 libnetfilter-contrack3 libnfnetlink0
  libnftables1 libnftnl11 nftables pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-doc rinse
  zfs-fuse | zfsutils firewall
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io iptables libip4tc2 libip6tc2 libnetfilter-contrack3 libnfnetlink0
  libnftables1 libnftnl11 nftables pigz runc ubuntu-fan
0 upgraded, 16 newly installed, 0 to remove and 2 not upgraded.
Need to get 74.8 MB of archives.
```

```
anushka@Anushka:/mnt/c/Users/anush$ docker version
Client:
 Version:           29.1.3
 API version:       1.52
 Go version:        go1.24.4
 Git commit:        29.1.3-0ubuntu3~24.04.1
 Built:             Fri Mar  6 11:35:12 2026
 OS/Arch:           linux/amd64
 Context:           default
```

PHASE 2 — START THE FABRIC NETWORK

Open a **brand new terminal (Terminal 1)**. Run the two commands below and keep this terminal open for the entire experiment. Do NOT close it.

Step 8: Launch Microfab (Terminal 1)

code:

export

MICROFAB_CONFIG='{ "endorsing_organisations": [{"name": "Org1"}], "channels": [{"name": "mango-channel", "endorsing_organisations": ["Org1"]}], "capability_level": "V2_0" }'

code:

docker run -e MICROFAB_CONFIG -p 8080:8080 ibmcom/ibp-microfab

...

Wait until you see this line before continuing:

...

Microfab started in XXXX ms

```
{
  "port": 8080,
  "endorsing_organizations": [
    {
      "name": "ProducersOrg"
    },
    {
      "name": "SellersOrg"
    }
  ],
  "channels": [
    {
      "name": "mango-channel",
      "endorsing_organizations": [
        "ProducersOrg",
        "SellersOrg"
      ]
    }
  ]
}
```

```
Unable to find image 'ibmcom/ibp-microfab:latest' locally
latest: Pulling from ibmcom/ibp-microfab
78f82eb67606: Download complete
b336a633f9e2: Download complete
b25deee7e50f: Download complete
9041dff50abb: Download complete
f24a19b34ee1: Download complete
c50c12d7db31: Download complete
576db8757925: Download complete
5186d5863148: Download complete
734dce7f474b: Download complete
eb98c0bd8a72: Download complete
65606c3065d6: Download complete
690939f6038f: Download complete
2785a9abab40: Download complete
d6e3788a121c: Download complete
531dcea8d07e: Download complete
Digest: sha256:125295f59f5be6c719141999a42b584ba5dcf7e388aa51c2c8d12cb2d2ca5859
Status: Downloaded newer image for ibmcom/ibp-microfab:latest
[    microfabd] 2026/04/03 13:30:49 Starting Microfab ...
[    microfabd] 2026/04/03 13:30:49 MRW
```

Hyperledger Fabric network is fully bootstrapped

```
[    sellersorgpeer] 2026-04-03 13:30:51.921 UTC 003a INFO [gossip.comm] GossipStream -> Peer 576d81a0d8d
48ce669212ad1153a4e1c115ae17edcedddad54007509f4c7dc9 ([::1]:59946) probed us
[    sellersorgpeer] 2026-04-03 13:30:51.921 UTC 003b INFO [comm.grpc.server] 1 -> streaming call complet
d grpc.service=gossip.Gossip grpc.method=GossipStream grpc.request_deadline=2026-04-03T13:31:01.918Z gr
c.peer_address=[::1]:59946 grpc.code=OK grpc.call_duration=2.792718ms
[    sellersorgpeer] 2026-04-03 13:30:51.939 UTC 003c INFO [comm.grpc.server] 1 -> unary call completed g
pc.service=gossip.Gossip grpc.method=Ping grpc.request_deadline=2026-04-03T13:30:53.939Z grpc.peer_addre
ss=[::1]:59946 grpc.code=OK grpc.call_duration=111.991µs
[producersorgpeer] 2026-04-03 13:30:52.024 UTC 0039 INFO [comm.grpc.server] 1 -> unary call completed g
pc.service=gossip.Gossip grpc.method=Ping grpc.request_deadline=2026-04-03T13:30:54.023Z grpc.peer_addre
ss=[::1]:49786 grpc.code=OK grpc.call_duration=131.73µs
[producersorgpeer] 2026-04-03 13:30:52.027 UTC 003a INFO [gossip.comm] GossipStream -> Peer e5edb1560729
b17aa870ed0aff036986f7434f0a987d18c3655973758b754b12 ([::1]:49786) probed us
[producersorgpeer] 2026-04-03 13:30:52.027 UTC 003b INFO [comm.grpc.server] 1 -> streaming call complet
d grpc.service=gossip.Gossip grpc.method=GossipStream grpc.request_deadline=2026-04-03T13:31:02.026Z gr
c.peer_address=[::1]:49786 grpc.code=OK grpc.call_duration=1.324849ms
[    sellersorgpeer] 2026-04-03 13:30:55.832 UTC 003d INFO [gossip.channel] reportMembershipChanges -> [[m
ango-channel] Membership view has changed. peers went online: [[producersorgpeer-api.127-0-0-1.nip.io:80
080 ]] , current view: [[producersorgpeer-api.127-0-0-1.nip.io:8080 ]]]
[producersorgpeer] 2026-04-03 13:30:55.839 UTC 003c INFO [gossip.channel] reportMembershipChanges -> [[m
ango-channel] Membership view has changed. peers went online: [[sellersorgpeer-api.127-0-0-1.nip.io:80
0 ]] , current view: [[sellersorgpeer-api.127-0-0-1.nip.io:8080 ]]]
```

PHASE 3 — PROJECT SETUP

Step 9: Create project directory

```
code:
cd ~
mkdir KBA-Mango
cd KBA-Mango
```

Step 10: Download Hyperledger Fabric binaries

```
code:
curl -sSL https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh |
code: -s -- binary
```

Wait for it to finish. Then verify:

```
code:
ls bin/
```

Expected: you should see files like peer, orderer, fabric-ca-client etc.

```
root@snehal:~# su - myuser
myuser@snehal:~$ cd ~
mkdir KBA-Mango
cd KBA-Mango
myuser@snehal:~/KBA-Mango$

ledger/fabric/main/scripts/install-fabric.sh | bash -s -- binary

Pull Hyperledger Fabric binaries

====> Downloading version 2.5.15 platform specific fabric binaries
====> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
====> Will unpack to: /home/myuser/KBA-Mango
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curr
ent                                     Dload  Upload  Total  Spent    Left  Spee
d
  0     0     0     0     0     0      0      0 --:--:-- --:--:-- --:--:--
  0     0     0     0     0     0      0      0 --:--:-- --:--:-- --:--:--
  0     0     0     0     0     0      0      0 --:--:~ --:~:~ --:~:~
0
  0 120M   0 32750   0     0 20612      0 1:42:35 0:00:01 1:42:34 206
  7 120M   7 9487k   0     0 4113k      0 0:00:30 0:00:02 0:00:28 12.
 17 120M  17 21.4M   0     0 6637k      0 0:00:18 0:00:03 0:00:15 12.
 27 120M  27 33.3M   0     0 7910k      0 0:00:15 0:00:04 0:00:11 12.
 33 120M  33 40.0M   0     0 7700k      0 0:00:16 0:00:05 0:00:11 10.
 41 120M  41 50.0M   0     0 7751k      0 0:00:15 0:00:06 0:00:09 9.
 48 120M  48 58.9M   0     0 8262k      0 0:00:14 0:00:07 0:00:07 9.
 59 120M  59 71.4M   0     0 8806k      0 0:00:14 0:00:08 0:00:06 9.
 69 120M  69 84.0M   0     0 9243k      0 0:00:13 0:00:09 0:00:04 10.
 79 120M  79 96.6M   0     0 9599k      0 0:00:12 0:00:10 0:00:02 11.
 90 120M  90 109M   0     0 9890k      0 0:00:12 0:00:11 0:00:01 12.
 95 120M  95 114M   0     0 9565k      0 0:00:12 0:00:12 --:~:~ --:~:~ 11.
100 120M 100 120M   0     0 9675k      0 0:00:12 0:00:12 --:~:~ --:~:~ 11.
0M
====> Done.
====> Downloading version 1.5.17 platform specific fabric-ca-client binary
```

% Total ent	% Received		% Xferd		Average Speed		Time	Time	Time	Curr
					Dload	Upload	Total	Spent	Left	Spee
d										
0	0	0	0	0	0	0	0	--:--:--	--:--:--	--:--:--
0	0	0	0	0	0	0	0	--:--:--	--:--:--	--:--:--
0										
0	0	0	0	0	0	0	0	--:--:--	--:--:--	--:--:--
9	31.6M	9	3199k	0	0	2163k	0	0:00:14	0:00:01	0:00:13 323
36	31.6M	36	11.4M	0	0	4714k	0	0:00:06	0:00:02	0:00:04 587
74	31.6M	74	23.4M	0	0	6897k	0	0:00:04	0:00:03	0:00:01 802
100	31.6M	100	31.6M	0	0	7455k	0	0:00:04	0:00:04	--:--:-- 840
0k										
==> Done.										

```
configtxgen cryptogen fabric-ca-client ledgerutil osnadmin
configtxlator discover fabric-ca-server orderer peer
```

Step 11: Set all environment variables

Run all of these one by one:

code:

```
export PATH=$PATH:${PWD}/bin
export FABRIC_CFG_PATH=${PWD}/config
export GOPATH=$HOME/go
export PATH=$PATH:$GOPATH/bin
```

Verify peer binary works:

code:

peer version

```
export FABRIC_CFG_PATH=${PWD}/config
export GOPATH=$HOME/go
export PATH=$PATH:$GOPATH/bin
```

```
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger
```

Step 12: Generate wallets, gateway, and MSP

code:

```
curl -s http://console.127-0-0-1.nip.io:8080/ak/api/v1/components | weft microfab -w ./_wallets
-p ./_gateways -m ./_msp -f
...
```

You will see output like this — copy the `export` lines shown for `org1admin`:

...

For org1admin @ Org1 use these:

```
export CORE_PEER_LOCALMSPID=Org1MSP
export
CORE_PEER_MSPCONFIGPATH=/home/<user>/KBA-Mango/_msp/Org1/org1admin/msp
export CORE_PEER_ADDRESS=org1peer-api.127-0-0-1.nip.io:8080
```

```
pi/v1/components | weft microfab -w ./_wallets -p ./_gateways -m ./_msp -f
Gateway profile written to : /home/myuser/KBA-Mango/_gateways/producersorggateway.json
Gateway profile written to : /home/myuser/KBA-Mango/_gateways/sellersorggateway.json
Added identity under label sellersorgadmin to the wallet at /home/myuser/KBA-Mango/_wallets/SellersOrg
Added identity under label sellersorgcaadmin to the wallet at /home/myuser/KBA-Mango/_wallets/SellersOrg
Added identity under label ordereradmin to the wallet at /home/myuser/KBA-Mango/_wallets/Orderer
```

```
For producersorgcaadmin @ ProducersOrg use these:

export CORE_PEER_LOCALMSPID=ProducersOrgMSP
export CORE_PEER_MSPCONFIGPATH=/home/myuser/KBA-Mango/_msp/ProducersOrg/producersorgcaadmin/msp
export CORE_PEER_ADDRESS=producersorgpeer-api.127-0-0-1.nip.io:8080
```

```
For sellersorgadmin @ SellersOrg use these:

export CORE_PEER_LOCALMSPID=SellersOrgMSP
export CORE_PEER_MSPCONFIGPATH=/home/myuser/KBA-Mango/_msp/SellersOrg/sellersorgadmin/msp
export CORE_PEER_ADDRESS=sellersorgpeer-api.127-0-0-1.nip.io:8080
```

```
For sellersorgcaadmin @ SellersOrg use these:

export CORE_PEER_LOCALMSPID=SellersOrgMSP
export CORE_PEER_MSPCONFIGPATH=/home/myuser/KBA-Mango/_msp/SellersOrg/sellersorgcaadmin/msp
export CORE_PEER_ADDRESS=sellersorgpeer-api.127-0-0-1.nip.io:8080
```

Step 13: Set peer environment variables

Run exactly the three lines shown in the output above. They will look like this (use YOUR actual username, not the example):

code:

```
export CORE_PEER_LOCALMSPID=Org1MSP
export
CORE_PEER_MSPCONFIGPATH=/home/<your-username>/KBA-Mango/_msp/Org1/org1admin/msp
export CORE_PEER_ADDRESS=org1peer-api.127-0-0-1.nip.io:8080
```

If you don't know your username:

```
export CORE_PEER_MSPCONFIGPATH=/home/myuser/KBA-Mango/_msp/ProducersOrg/producersorgadmin/msp
export CORE_PEER_ADDRESS=producersorgpeer-api.127-0-0-1.nip.io:8080
```

code:

whoami

Verify the MSP folder exists:

code:

```
ls ./_msp/Org1/org1admin/msp
```

Expected: admincerts cacerts keystore signcerts

```
myuser@snehal:~/KBA-Mango$ ls ./_msp/ProducersOrg/producersorgadmin/msp
admincerts  cacerts  keystore  signcerts
```

PHASE 4 — WRITE THE CHAINCODE

Step 14: Create the chaincode directory and initialise Go module

code:

```
cd ~/KBA-Mango
mkdir KBA-Mango-Contract
cd KBA-Mango-Contract
go mod init mango
```

Create the Chaincode File

```
touch mangoContract.go
nano mangoContract.go
```

```
myuser@snehal:~/KBA-Mango$ cd ~/KBA-Mango
mkdir KBA-Mango-Contract
cd KBA-Mango-Contract
```

```
cd KBA-Mango-Contract
myuser@snehal:~/KBA-Mango/KBA-Mango-Contract$ go mod init mango
go: creating new go.mod: module mango
```

```
fabric-contract-api-go@v1.2.1
go: downloading github.com/hyperledger/fabric-contract-api-go v1.2.1
go: added github.com/hyperledger/fabric-contract-api-go v1.2.1
myuser@snehal:~/KBA-Mango/KBA-Mango-Contract$ go get google.golang.org/grpc@
v1.56.3
go: downloading google.golang.org/grpc v1.56.3
go: downloading golang.org/x/net v0.9.0
go: downloading github.com/golang/protobuf v1.5.3
```

```

1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6     "github.com/hyperledger/fabric-contract-api-go/contractapi"
7 )
8
9 type MangoContract struct {
10     contractapi.Contract
11 }
12
13 type Mango struct {
14     ID          string `json:"ID"`
15     BatchNumber int    `json:"BatchNumber"`
16     Producer    string `json:"Producer"`
17     OwnedBy     string `json:"OwnedBy"`
18     Quantity    int    `json:"Quantity"`
19     Price       float32 `json:"Price"`
20 }
21
22 func (c *MangoContract) MangoExists(ctx contractapi.TransactionContextInterface, mangoID string) (bool, error) {
23     data, err := ctx.GetStub().GetState(mangoID)
24     if err != nil {
25         return false, err
26     }
27     return data != nil, nil
28 }
29
30 func (c *MangoContract) CreateMango(ctx contractapi.TransactionContextInterface, mangoID string, batchNum
31     exists, err := c.MangoExists(ctx, mangoID)
32     if err != nil {
33         return fmt.Errorf("Could not read from world state. %s", err)
34     } else if exists {
35         return fmt.Errorf("The asset %s already exists", mangoID)
36     }

```

Step 15: Write the chaincode file

Paste the complete chaincode below using Ctrl+Shift+V. Then save with Ctrl+O, Enter. Exit with Ctrl+X.

code:

```
package main
```

```
import (
    "encoding/json"
    "fmt"
    "github.com/hyperledger/fabric-contract-api-go/contractapi"
)
```

// MangoContract defines the smart contract structure

```
type MangoContract struct {
    contractapi.Contract
}
```

// Mango defines the asset structure stored on the ledger

```
type Mango struct {
    ID          string `json:"ID"`
```

```

    BatchNumber int    `json:"BatchNumber"`
    Producer    string `json:"Producer"`
    OwnedBy     string `json:"OwnedBy"`
    Quantity    int    `json:"Quantity"`
    Price       float32 `json:"Price"`
}

// MangoExists checks if a mango asset exists in the world state
func (c *MangoContract) MangoExists(ctx contractapi.TransactionContextInterface, mangoID
string) (bool, error) {
    data, err := ctx.GetStub().GetState(mangoID)
    if err != nil {
        return false, err
    }
    return data != nil, nil
}

// CreateMango adds a new mango asset to the ledger
func (c *MangoContract) CreateMango(ctx contractapi.TransactionContextInterface, mangoID
string, batchNumber int, producer string, quantity int, price float32) error {
    exists, err := c.MangoExists(ctx, mangoID)
    if err != nil {
        return fmt.Errorf("Could not read from world state. %s", err)
    } else if exists {
        return fmt.Errorf("The asset %s already exists", mangoID)
    }
    mango := Mango{
        ID:         mangoID,
        BatchNumber: batchNumber,
        Producer:   producer,
        OwnedBy:    producer,
        Quantity:   quantity,
        Price:      price,
    }
    bytes, _ := json.Marshal(mango)
    return ctx.GetStub().PutState(mangoID, bytes)
}

// ReadMango retrieves a mango asset from the ledger

```

```

func (c *MangoContract) ReadMango(ctx contractapi.TransactionContextInterface, mangoID
string) (*Mango, error) {
    exists, err := c.MangoExists(ctx, mangoID)
    if err != nil {
        return nil, fmt.Errorf("Could not read from world state. %s", err)
    } else if !exists {
        return nil, fmt.Errorf("The asset %s does not exist", mangoID)
    }
    bytes, _ := ctx.GetStub().GetState(mangoID)
    mango := new(Mango)
    err = json.Unmarshal(bytes, mango)
    if err != nil {
        return nil, fmt.Errorf("Could not unmarshal world state data to type Mango")
    }
    return mango, nil
}

```

// UpdateMango updates an existing mango asset on the ledger

```

func (c *MangoContract) UpdateMango(ctx contractapi.TransactionContextInterface, mangoID
string, batchNumber int, producer string, owner string, quantity int, price float32) error {
    exists, err := c.MangoExists(ctx, mangoID)
    if err != nil {
        return fmt.Errorf("Could not read from world state. %s", err)
    } else if !exists {
        return fmt.Errorf("The asset %s does not exist", mangoID)
    }
    mango := Mango{
        ID:      mangoID,
        BatchNumber: batchNumber,
        Producer: producer,
        OwnedBy:  owner,
        Quantity: quantity,
        Price:    price,
    }
    bytes, _ := json.Marshal(mango)
    return ctx.GetStub().PutState(mangoID, bytes)
}

```

// DeleteMango removes a mango asset from the ledger

```

func (c *MangoContract) DeleteMango(ctx contractapi.TransactionContextInterface, mangoID
string) error {
    exists, err := c.MangoExists(ctx, mangoID)
    if err != nil {
        return fmt.Errorf("Could not read from world state. %s", err)
    } else if !exists {
        return fmt.Errorf("The asset %s does not exist", mangoID)
    }
    return ctx.GetStub().DelState(mangoID)
}

```

// SellMango transfers ownership of a mango asset to a new owner

```

func (c *MangoContract) SellMango(ctx contractapi.TransactionContextInterface, mangoID
string, owner string, newOwner string) error {
    exists, err := c.MangoExists(ctx, mangoID)
    if err != nil {
        return fmt.Errorf("Could not read from world state. %s", err)
    } else if !exists {
        return fmt.Errorf("The asset %s does not exist", mangoID)
    }
    mango, _ := c.ReadMango(ctx, mangoID)
    if mango.OwnedBy == owner {
        mango.OwnedBy = newOwner
        bytes, _ := json.Marshal(mango)
        return ctx.GetStub().PutState(mangoID, bytes)
    } else {
        return fmt.Errorf("Asset is not owned by %v, only original owner can sell the
asset", owner)
    }
}

```

```

func main() {
    mangoContract := new(MangoContract)
    chaincode, err := contractapi.NewChaincode(mangoContract)
    if err != nil {
        panic("Could not create chaincode." + err.Error())
    }
    err = chaincode.Start()
    if err != nil {
        panic("Failed to start chaincode. " + err.Error())
    }
}

```

```

    }
}

```

PHASE 5 — FIX DEPENDENCIES

Microfab builds chaincode internally using Go 1.18. Default pulled dependencies require Go 1.19+. These exact pinned versions are the ones that work.

Step 16: Pull the main fabric dependency first

code:

```
go get github.com/hyperledger/fabric-contract-api-go@v1.2.1
```

Step 17: Pin gRPC to a Go 1.18 compatible version

code:

```
go get google.golang.org/grpc@v1.56.3
```

```

go get github.com/go-openapi/jsonreference@v0.20.2
go get github.com/go-openapi/swag@v0.22.4
go: downloading github.com/go-openapi/jsonpointer v0.19.6
go: downloading github.com/go-openapi/swag v0.22.3
go: downloading github.com/mailru/easyjson v0.7.7
go: downloading gopkg.in/yaml.v3 v3.0.1
go: downloading github.com/josharian/intern v1.0.0
go: upgraded github.com/go-openapi/jsonpointer v0.19.5 => v0.19.6
go: upgraded github.com/go-openapi/swag v0.21.1 => v0.22.3
go: downloading github.com/go-openapi/jsonreference v0.20.2

```

Step 18: Pin openapi dependencies to Go 1.18 compatible versions

code:

```

go get github.com/go-openapi/jsonpointer@v0.19.6
go get github.com/go-openapi/jsonreference@v0.20.2
go get github.com/go-openapi/swag@v0.22.4

```

```

go get github.com/go-openapi/jsonreference@v0.20.2
go get github.com/go-openapi/swag@v0.22.4
go: downloading github.com/go-openapi/jsonpointer v0.19.6
go: downloading github.com/go-openapi/swag v0.22.3
go: downloading github.com/mailru/easyjson v0.7.7
go: downloading gopkg.in/yaml.v3 v3.0.1
go: downloading github.com/josharian/intern v1.0.0
go: upgraded github.com/go-openapi/jsonpointer v0.19.5 => v0.19.6
go: upgraded github.com/go-openapi/swag v0.21.1 => v0.22.3
go: downloading github.com/go-openapi/jsonreference v0.20.2

```

Step 19: Tidy and build

code:

```
go mod tidy
```

```
go build ./...
```

```
go build ./...
go: downloading github.com/hyperledger/fabric-protos-go v0.3.0
go: downloading github.com/hyperledger/fabric-chaincode-go v0.0.0-20230228194215-b84622ba6a7a
go: downloading github.com/stretchr/testify v1.8.2
go: downloading github.com/go-openapi/spec v0.20.8
```

```
myuser@shane: ~/KBA-Mango/KBA-Mango-Contract$ go mod tidy
go build ./...
go: downloading github.com/hyperledger/fabric-protos-go v0.3.0
go: downloading github.com/hyperledger/fabric-chaincode-go v0.0.0-20230228194215-b84622ba6a7a
go: downloading github.com/stretchr/testify v1.8.2
go: downloading github.com/go-openapi/spec v0.20.8
go: downloading github.com/xeipuuv/gojsonschema v1.2.0
go: downloading github.com/gobuffalo/packr v1.30.1
go: downloading gopkg.in/yaml.v2 v2.4.0
go: downloading gopkg.in/check.v1 v1.0.0-20201130134442-10cb98267c6c
```

PHASE 6 — PACKAGE AND INSTALL CHAINCODE

Step 20: Go back to the project root

code:

```
cd ~/KBA-Mango
```

Step 21: Package the chaincode

code:

```
peer lifecycle chaincode package mango.tgz \
  --path ./KBA-Mango-Contract/ \
  --lang golang \
  --label mango_1
```

Verify the package was created:

```
myuser@snehal:~/KBA-Mango/KBA-Mango-Contract$ cd ~/KBA-Mango
myuser@snehal:~/KBA-Mango$ peer lifecycle chaincode package mango.tgz \
--path ./KBA-Mango-Contract/ \
--lang golang \
--label mango_1
```

Step 22: Install chaincode on the peer

code:

```
peer lifecycle chaincode install mango.tgz
```

Expected output:

INFO Installed remotely: response:<status:200 ...>

INFO Chaincode code package identifier: mango_1:<long-hash-string>

Copy the entire mango_1:xxxxxx... identifier from the output. You need it in the next step.

```
2026-04-03 16:53:43.757 UTC 0001 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Installed remotely: response:<status:200 payload:"\nHmango_1:01dc00b9bf1607543e76c991bff583d23847a19fc066071c0cff6871494cda2b\022\007mango_1" >
2026-04-03 16:53:43.758 UTC 0002 INFO [cli.lifecycle.chaincode] submitInstallProposal -> Chaincode code package identifier: mango_1:01dc00b9bf1607543e76c991bff583d23847a19fc066071c0cff6871494cda2b
```

```
-o orderer-api.127-0-0-1.nip.io:8080 \
--channelID mango-channel \
--name mango \
--version 1 \
--sequence 1 \
--waitForEvent \
--package-id ${CC_PACKAGE_ID}
2026-04-03 16:59:32.943 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [319ae6a457d5dbce7a07a14e63ae730f30454882597352a3e172dc55601f4ab2] committed with status (VALID) at producersorgpeer-api.127-0-0-1.nip.io:8080
```

Step 23: Export the Package ID

Paste your exact identifier from the output above:

code:

```
export CC_PACKAGE_ID=mango_1:<paste-your-exact-hash-here>
```

Verify it is set:

code:

```
echo $CC_PACKAGE_ID
```

Expected: prints your full package ID, not empty.

```
c991bfff583d23847a19fc066071c0cff6871494cda2b
myuser@snehal:~/KBA-Mango$ peer lifecycle chaincode approveformyorg \
-o orderer-api.127-0-0-1.nip.io:8080 \
--channelID mango-channel \
--name mango \
--version 1 \
--sequence 1 \
--waitForEvent \
--package-id ${CC_PACKAGE_ID}
```

PHASE 7 — APPROVE AND COMMIT

Step 24: Approve chaincode for Org1

code:

```
peer lifecycle chaincode approveformyorg \
-o orderer-api.127-0-0-1.nip.io:8080 \
--channelID mango-channel \
--name mango \
--version 1 \
--sequence 1 \
--waitForEvent \
--package-id ${CC_PACKAGE_ID}
```

Expected: txid [...] committed with status (VALID)

```
-o orderer-api.127-0-0-1.nip.io:8080 \
--channelID mango-channel \
--name mango \
--version 1 \
--sequence 1 \
--waitForEvent \
--package-id ${CC_PACKAGE_ID}
```

Step 25: Commit chaincode to the channel

code:

```
peer lifecycle chaincode commit \  
-o orderer-api.127-0-0-1.nip.io:8080 \  
--channelID mango-channel \  
--name mango \  
--version 1 \  
--sequence 1
```

Expected: txid [...] committed with status (VALID)

```
2026-04-03 16:59:32.943 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [319  
ae6a457d5dbce7a07a14e63ae730f30454882597352a3e172dc55601f4ab2] committed wit  
h status (VALID) at producersorgpeer-api.127-0-0-1.nip.io:8080  
muser@archal:~/UDA-Mango$
```

```
-o orderer-api.127-0-0-1.nip.io:8080 \  
--channelID mango-channel \  
--name mango \  
--version 1 \  
--sequence 1  
2026-04-03 17:01:16.359 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [f9c  
e54801ecb20d90d6e985e10d7dca2032c591c0b15b5f2907d3d4351a21a6a] committed wit  
h status (VALID) at producersorgpeer-api.127-0-0-1.nip.io:8080
```

PHASE 8 — TEST ALL CHAINCODE FUNCTIONS

Step 26: CreateMango

code:

```
peer chaincode invoke \  
-o orderer-api.127-0-0-1.nip.io:8080 \  
--channelID mango-channel \  
-n mango \  
-c '{"function": "CreateMango", "Args": ["MANGO1", "1", "mango corp", "5000", "10000"]}'
```

Expected: Chaincode invoke successful. result: status:200

```
-o orderer-api.127-0-0-1.nip.io:8080 \  
--channelID mango-channel \  
-n mango \  
-c '{"function":"CreateMango","Args":["MANGO1","1","mango corp","5000","10000"]}'  
2026-04-03 17:03:26.033 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery  
-> Chaincode invoke successful. result: status:200
```

Step 27: ReadMango

code:

```
peer chaincode query \  
-o orderer-api.127-0-0-1.nip.io:8080 \  
--channelID mango-channel \  
-n mango \  
-c '{"function":"ReadMango","Args":["MANGO1"]}'
```

Expected:

json

```
{"ID":"MANGO1","BatchNumber":1,"Producer":"mango corp","OwnedBy":"mango corp","Quantity":5000,"Price":10000}
```

```
-o orderer-api.127-0-0-1.nip.io:8080 \  
--channelID mango-channel \  
-n mango \  
-c '{"function":"ReadMango","Args":["MANGO1"]}'  
{ "ID": "MANGO1", "BatchNumber": 1, "Producer": "mango corp", "OwnedBy": "mango corp", "Quantity": 5000, "Price": 10000 }
```

Step 28: UpdateMango

code:

```
peer chaincode invoke \  
-o orderer-api.127-0-0-1.nip.io:8080 \  
--channelID mango-channel \  
-n mango \  
-c '{"function":"UpdateMango","Args":["MANGO1","2","mango corp","mango corp","6000","12000"]}'
```

Expected: Chaincode invoke successful. result: status:200

```
-o orderer-api.127-0-0-1.nip.io:8080 \  
--channelID mango-channel \  
-n mango \  
-c '{"function":"UpdateMango","Args":["MANGO1","2","mango corp","mango corp"  
,"6000","12000"]}'  
2026-04-03 17:04:46.382 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery  
-> Chaincode invoke successful. result: status:200
```

Step 29: SellMango

code:

peer chaincode invoke \

-o orderer-api.127-0-0-1.nip.io:8080 \

--channelID mango-channel \

-n mango \

-c '{"function":"SellMango","Args":["MANGO1","mango corp","mango shop"]}'

Expected: Chaincode invoke successful. result: status:200

```
-o orderer-api.127-0-0-1.nip.io:8080 \  
--channelID mango-channel \  
-n mango \  
-c '{"function":"SellMango","Args":["MANGO1","mango corp","mango shop"]}'  
2026-04-03 17:05:16.620 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery  
-> Chaincode invoke successful. result: status:200
```

Step 30: ReadMango — verify ownership transferred

code:

peer chaincode query \

-o orderer-api.127-0-0-1.nip.io:8080 \

--channelID mango-channel \

-n mango \

-c '{"function":"ReadMango","Args":["MANGO1"]}'

Expected:

```
{ "ID": "MANGO1", "BatchNumber": 2, "Producer": "mango corp", "OwnedBy": "mango  
shop", "Quantity": 6000, "Price": 12000 }
```

```
-c '{"function":"ReadMango","Args":["MANGO1"]}'
Error: The required parameter 'channelID' is empty. Rerun the command with -C flag
Usage:
  peer chaincode query [flags]

Flags:
  -C, --channelID string          The channel on which this command should be executed
  --connectionProfile string      Connection profile that provides the necessary connection information for the network. Note: currently only supported for providing peer connection information
  -c, --ctor string              Constructor message for the chaincode in JSON format (default "{}")
  -h, --help                     help for query
  -x, --hex                      If true, output the query value byte array in hexadecimal. Incompatible with --raw
  -n, --name string              Name of the chaincode
  --peerAddresses stringArray    The addresses of the peers to connect to
```

Step 31: DeleteMango

code:

```
peer chaincode invoke \
-o orderer-api.127-0-0-1.nip.io:8080 \
--channelID mango-channel \
-n mango \
-c '{"function":"DeleteMango","Args":["MANGO1"]}'
...
```

Expected: `Chaincode invoke successful. result: status:200`

```
-o orderer-api.127-0-0-1.nip.io:8080 \
--channelID mango-channel \
-n mango \
-c '{"function":"DeleteMango","Args":["MANGO1"]}'
2026-04-03 17:06:09.975 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery
-> Chaincode invoke successful. result: status:200
```

PHASE 9 — CLEANUP

Step 32: Stop Microfab

Go to Terminal 1 and press `Ctrl+C`. Wait for:

Microfab stopped

Step 33: Remove stopped containers

code:

`docker container prune`

Type y and press Enter.

```
anushka@Anushka:/mnt/c/Users/anush$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] |
```

```
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
df15175e23e884122621fe73740e055cc6d911262859cb7d704780b504303fc9
5a38a7d6fceb559c8c5fef1c17b92c9e32879289dddbf7643966459f0c61ff5c
```

Conclusion : The Hyperledger Fabric network was successfully implemented, and chaincode was deployed to manage assets. Asset creation, querying, and transfer operations were performed efficiently, demonstrating secure, transparent, and reliable transaction handling in a permissioned blockchain system.